

28-Feb-2028

## Async Javascript

Date  
Page

Kahani

| Abhinav  | Abhinav V2 | outsourcing |
|----------|------------|-------------|
| chopping | Samosa     |             |
| masala   | momo       |             |
| fry      | serve      |             |
| gravy    | chopping   |             |
| serve    | fry        |             |

→ ye ek restaurant hai or abhinav ek chef hai or vo pura kitchen sambhalta hai

- chopping, masala, fry, gravy, serve

ye sare kam abhinav ke hi karne hai

- abhinav ko jo kam nahi karne vo outsourcing ko de deta hai

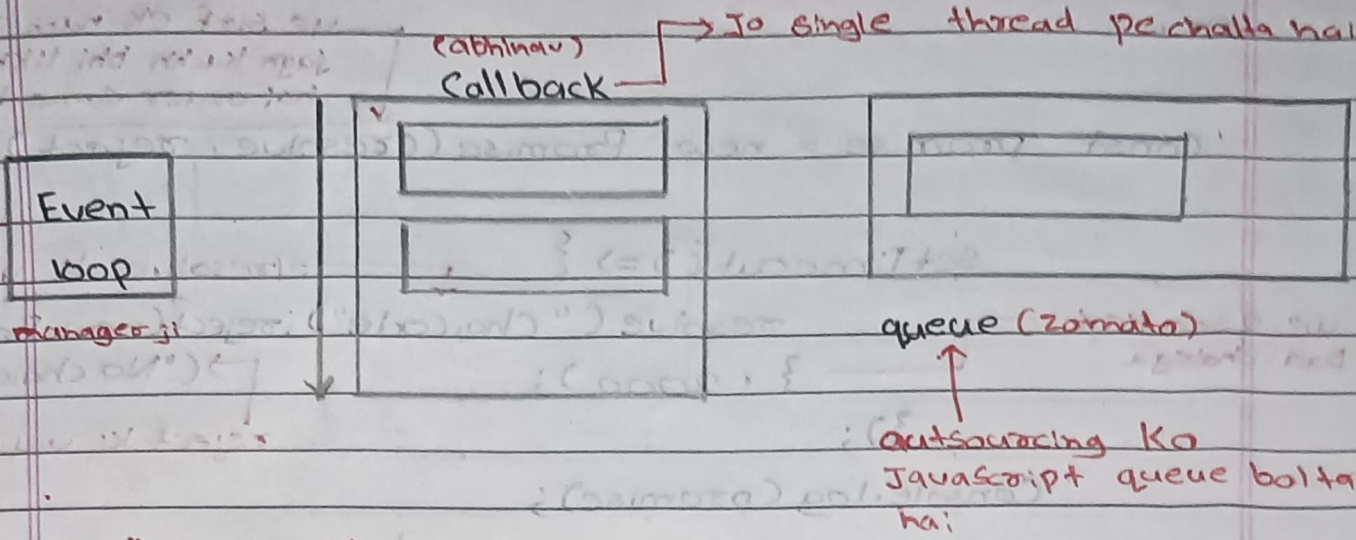
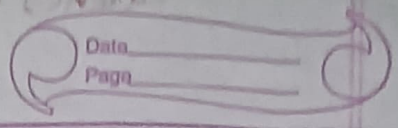
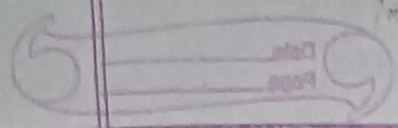
- or jab outsourcing me kam ho jata hai vo abhinav ko de deta hai

Yeha hi same kam javascript ke under hota hai

- Javascript single thread language

or uske pass bhi outsourcing hai, jitne kam use karne vo karta baki ka outsourcing ko pakda deta, par chata ek hi thread hai





ye dhyan rakhta hai ki callback

me next kya jayega, apke code file ko padhta hai

- Jo kam hame karna hai kareenge baki queue ko de denge, iska kam execute karke wapis dena hai

- callstack jo aaya hai bana do, ek hi kam hai eska run, execute kardo.

## 20 - Promises

### State of Promises

- pending
- done (Fulfill, resolve)
- nope (not, reject, nako)
- Promise ko new keyword lagake declared karta hai
- By default promise pending hota hai.
- setTimeout() delay introduced karta hai. apko jo kam karna hai thode der me kar dunga.



note:- `.then()` me resolve handle hote hai  
`.catch()` me error handle hote hai  
rejections

Date \_\_\_\_\_  
Page \_\_\_\_\_

ex. `const promise = new Promise((resolve, reject) => {`  
ye sirf ek nam hai inke jagh kuchh bhi likh sakte hai or der same hote hai

`resolve("chaicode"); reject(new Error("No chaicode"));`

`setTimeout(() => {`

ye 2sec ke bad chalega

`resolve("chaicode"); reject(new Error("No chaicode"));`

`});`

`console.log(promise);`

resolve ki value

reject ki value.

`promise.then((value) => {`  
Kuchh bhi nam de sakte ho

`console.log(value);`

`});`

callback.

`.then()` ke ander callback milta hai, function ek.

Jo bhi value resolve hoti hai, vo us `.then()` me insert ki jati hai

`promise.then(console.log);`

ek reference hai jis kar do

`setTimeout(() => {`

`console.log(promise);`

`}, 3000);`

ye time ek hi hai

`.then()` ke ander do parameter hote hai

Ek resolve hua to print kare or dusra error aaye

`promise.then(`  
`(data) => console.log(data), (error) => console.log(error),`  
`);`

OR



Business logic :- data aaya to .then() se resolve ki value dena,  
nahi to .catch() se error

Limit of .then()  $\Rightarrow 45,000$

45,000 + 1 pe fail ho jata hai

- .catch() modern syntax me aata hai

Promise

.then((data)  $\Rightarrow$  console.log(data))

.catch((error)  $\Rightarrow$  console.log(error));

- logic catch se return karna

(convert of aarest hai)

promise.then()

.then((data)  $\Rightarrow$  {

newData = data.toUpperCase();

return newData;

})

.then((data)  $\Rightarrow$  {

return data + ".com";

})

.then(console.log)

.catch((error)  $\Rightarrow$  {

console.log(error);

return "chai";

})

.then(console.log);

.then() ke  
under kitne  
bhi .then()

use krskr hai

catch se return

karna

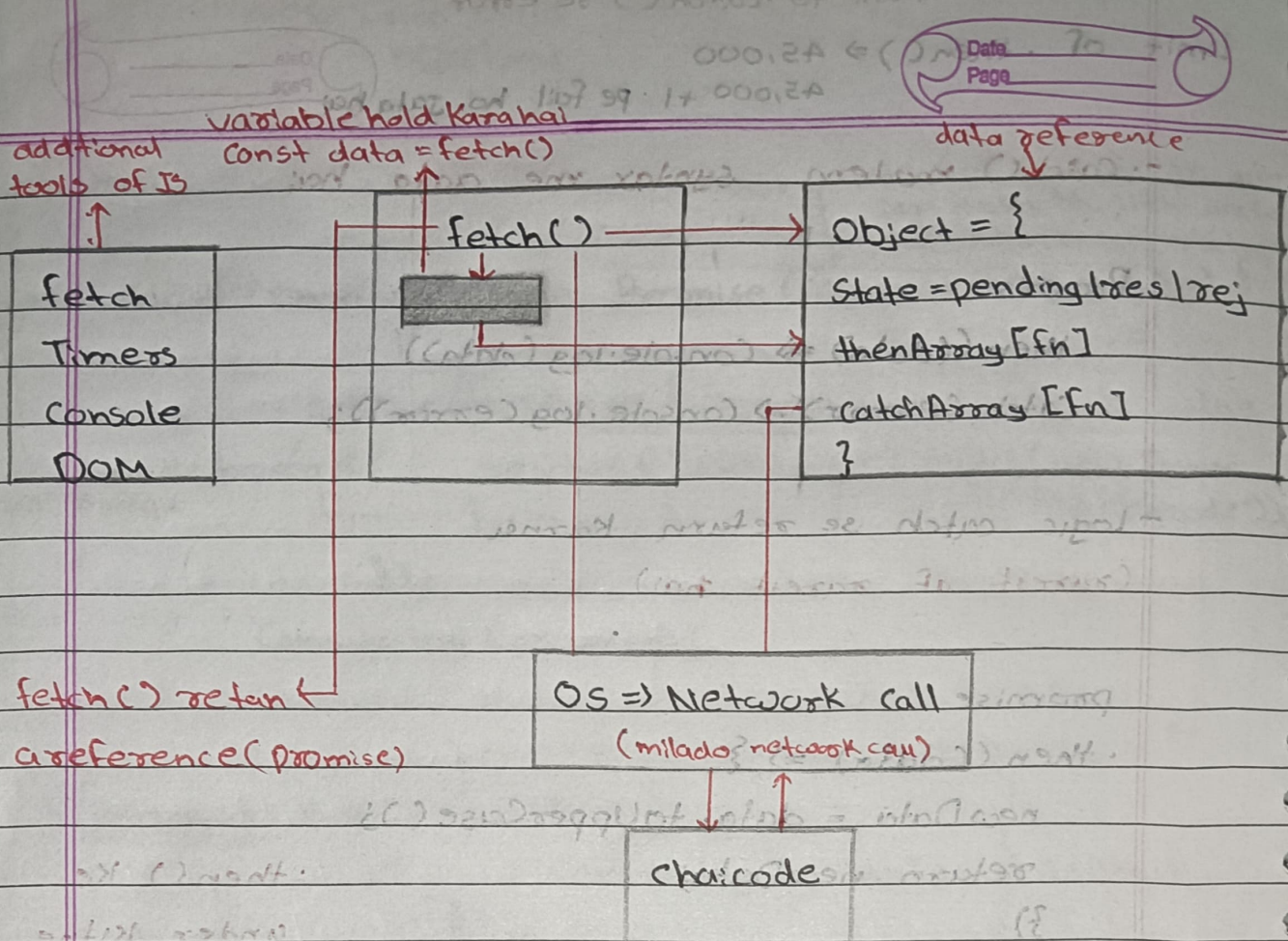
Output

Error: chaiCode

chai



Timers = setTimeout



- fetch() ke under jo data hoga ye kaise le paoge job uska reference hoga apke pass me.
- thenArray => Function ka Array iske under jitne chahi utne function rakho iska primary job hai, jo bhi data mere pass aaye, us data ko [fn] ke under.
- catchArray ka kam hai, jo bhi data aayega us data ko is function ke under [fn] inject kar do. argument ki tarah bhej do.
- fetch() javascript ka part nahi hai, promise hai. Browser ya javascript web call nahi kar sakti hai.
- JavaScript ke under external tool ne usi ke help se web call kiye jate hai, inko hi web API bolte hai.



meta programming It's mean number of ways a program can manipulate itself.

Date \_\_\_\_\_  
Page \_\_\_\_\_

- 1) return a reference
- 2) create this entire object or data reference,
- 3) OS gaya kisi server ke pass me.
- 4) ex, OS gaya charcode pe data leke aaya.
- 5) data ek to then ke under error agar error aaya to catch ke under update hoga.
- 6) fir state dega jese pending, resolve, reject
- 7) fir then function [fn] ke under vo data gaya.

code example.

```
const tenant = Promise.resolve("Tenant");
```

```
console.log(tenant);
```

Method in Promise

1) `Promise.all()`

- when all of the promises fulfill & rejects, when any of the promises rejects.

2) `Promise.allSettled()`

- resolved when all promises and settle

3) `Promise.race()`

- It settle as soon as first promise is settles, resolve or rejects

4) `Promise.any()`

- It resolved when any of the promises fulfills and rejects when all of the promises rejects



## async & await

- await abhi hi use kar sakte hai jab function asynchronous ho.

ex.

```
const hPromise = new Promise((res, rej) => {
  setTimeout(() => {
    res("Masterji");
  }, 3000);
});
```

async function nice() {

const result = await hPromise;

console.log(result);

nice()

→ 3 sec tak ruk ja  
→ 2 sec ke bad chal jaye

output

masterji

```
const newResult = await hPromise;
```

```
console.log(newResult);
```

output

masterji

- Recent edition, await ko globally bhi use kar sakte hai

(nahi use karne ka)

await me error handle nhi kar sakte.  
uske jgh try catch use karo

Date \_\_\_\_\_

Page \_\_\_\_\_

In case of rejection

```
(const hPromise = new Promise ((res, rej) => {  
  setTimeout(() => {  
    rej(new Error("Masterji"));  
  }, 3000);  
}));
```

```
async function nice() {
```

```
  try {
```

```
    const result = await hPromise;
```

```
    console.log(result);
```

```
  } catch (error) {
```

```
    console.log("Error aa gya ji", error);
```

```
  }
```

```
}
```

error me  
rej value  
bhejta  
hai

output

~~Error aa gya ji masterji~~

Error aa gya ji Error: masterji

ex,

```
console.log("Swastik"); ← sync
```

```
Promise.resolve("resolved value").then((v) => {
```

```
  console.log("Microtask", v); ← promise  
                                microtask.
```

```
});
```

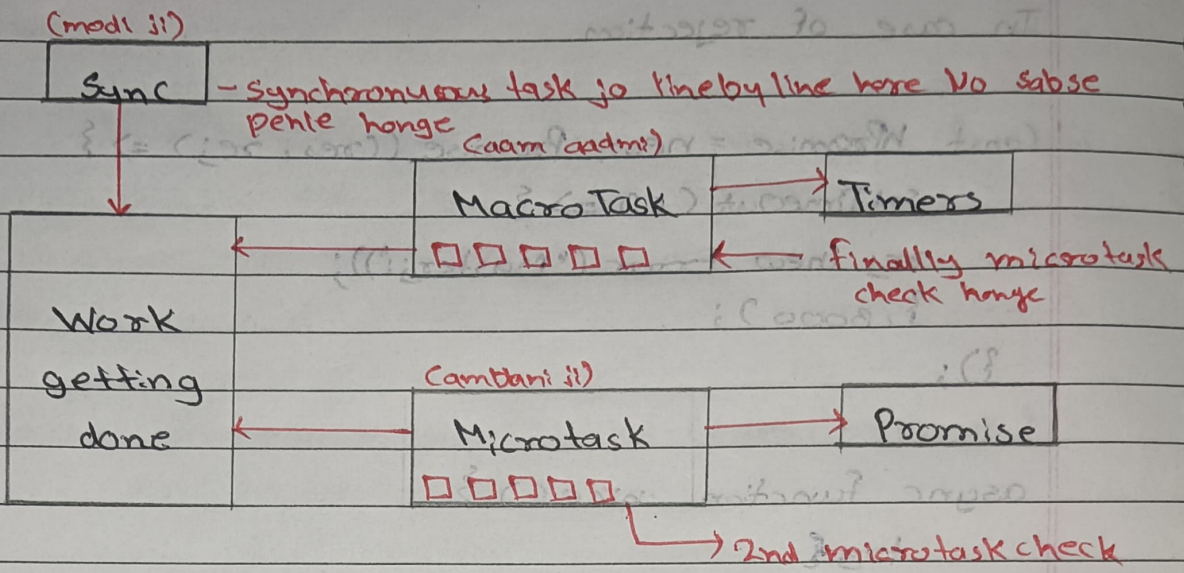
```
console.log("Avishek"); ← sync
```

output => Swastik

Avishek

Microtask resolved value.





Note: Sabse pehle sync karo synchronous ko hota do execute kardo fir promise microtask, jab timers macrotask.